

JavaScript + React Syntax Cheat-Sheet

A quick reference with syntax, mini-example & plain-English.

Variable Declaration

Syntax:

```
let variable = value;  
const variable = value;  
var variable = value;
```

Example:

```
const name = "Alice";  
let count = 0;
```

What it does: Creates a variable. `const` can't be reassigned, `let` can.

Function Declaration

Syntax:

```
function functionName(params) {  
  // code  
}
```

Example:

```
function greet(name) {  
  return "Hello, " + name;  
}
```

What it does: Defines a reusable block of code.

Arrow Function

Syntax:

```
const func = (params) => {  
  // code  
};
```

Example:

```
const add = (a, b) => a + b;
```

What it does: Shorter way to write functions, especially for callbacks.

Import / Export

Syntax:

```
import name from "module";  
import { thing } from "module";  
export default value;  
export { name };
```

Example:

```
import React from "react";  
export default MyComponent;
```

What it does: Shares code between files.

JSX Element

Syntax:

```
<tagName prop={value}>Content</tagName>
```

Example:

```
<h1>Hello, world!</h1>  
<button disabled={true}>Click</button>
```

What it does: Describes UI using HTML-like syntax in JavaScript.

Function Component

Syntax:

```
function ComponentName(props) {  
  return <JSX />;  
}
```

Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

What it does: Defines a reusable UI piece as a function.

Props (Component Inputs)

Syntax:

```
<Component propName={value} />  
function Comp(props) { props.propName }
```

Example:

```
<Greeting name="Alice" />
function Greeting(props) {
  return <p>Hello, {props.name}</p>;
}
```

What it does: Passes data into components.

State Hook (useState)

Syntax:

```
const [state, setState] = useState(initialValue);
```

Example:

```
const [count, setCount] = useState(0);
```

What it does: Adds local, changeable data to a function component.

Event Handling

Syntax:

```
<element onEvent={handler} />
```

Example:

```
<button onClick={handleClick}>Click</button>
```

What it does: Runs code when the user interacts with the UI.

Conditional Rendering

Syntax:

```
{condition && <JSX />}
{condition ? <A /> : <B />}
```

Example:

```
{isLoggedIn && <LogoutButton />}
{count > 0 ? <p>Yes</p> : <p>No</p>}
```

What it does: Shows or hides UI based on logic.

List Rendering (.map)

Syntax:

```
array.map(item => <JSX key={item.id} />)
```

Example:

```
<ul>
  {items.map(i => <li key={i.id}>{i.text}</li>)}
</ul>
```

What it does: Creates UI for each item in a list.

Effect Hook (useEffect)

Syntax:

```
useEffect(() => {
  // code
}, [dependencies]);
```

Example:

```
useEffect(() => {
  document.title = count;
}, [count]);
```

What it does: Runs code after render or when dependencies change.

Object Destructuring

Syntax:

```
const {a, b} = obj;
```

Example:

```
const {name, age} = person;
```

What it does: Pulls out properties from objects into variables.

Array Destructuring

Syntax:

```
const [a, b] = arr;
```

Example:

```
const [first, second] = [1, 2];
```

What it does: Pulls out values from arrays into variables.

Spread and Rest (...)

Syntax:

```
const arr2 = [...arr1, value];
const obj2 = {...obj1, key: value};
function fn(...args) {}
```

Example:

```
const all = [...nums, 4];
const copy = {...person, age: 21};
function sum(...nums) { }
```

What it does: Expands or collects items in arrays/objects/functions.

Class Component

Syntax:

```
class MyComponent extends React.Component {
  render() {
    return <JSX />;
  }
}
```

Example:

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

What it does: Defines a component using ES6 classes.

Class State

Syntax:

```
this.state = { key: value };
this.setState({ key: value });
```

Example:

```
this.state = { count: 0 };
this.setState({ count: 1 });
```

What it does: Stores and updates local data in class components.

Default Props

Syntax:

```
Component.defaultProps = { prop: value };
```

Example:

```
Greeting.defaultProps = { name: "Guest" };
```

What it does: Sets fallback values for props.

Fragments

Syntax:

```
<>
  <A />
  <B />
</>
```

Example:

```
<>
  <li>One</li>
  <li>Two</li>
</>
```

What it does: Groups elements without adding extra DOM nodes.

Inline Styles

Syntax:

```
<element style={{ key: value }} />
```

Example:

```
<div style={{ color: "red", fontSize: 18 }} />
```

What it does: Adds CSS directly to elements using objects.

Comments

Syntax:

```
// Single line
/* Multi-line */
{/* JSX comment */}
```

Example:

```
// This is a comment
```

```
{/* Only in JSX */}
```

What it does: Adds notes for humans; ignored by JS/React.

Promises and Async/Await

Syntax:

```
fetch(url).then(res => res.json());  
async function fn() { await something(); }
```

Example:

```
async function getData() {  
  const res = await fetch("/api");  
}
```

What it does: Handles asynchronous code (like data fetching).

Module Exports (CommonJS)

Syntax:

```
module.exports = value;  
const thing = require("module");
```

Example:

```
module.exports = MyComponent;  
const fs = require("fs");
```

What it does: Shares code in Node.js (non-ESM).

Array Methods

Syntax:

```
arr.map(fn)  
arr.filter(fn)  
arr.reduce(fn, init)  
arr.forEach(fn)
```

Example:

```
const evens = nums.filter(n => n % 2 === 0);
```

What it does: Processes arrays in various ways.

Object Methods

Syntax:

```
Object.keys(obj)
Object.values(obj)
Object.entries(obj)
```

Example:

```
Object.keys({a:1, b:2}) // ["a", "b"]
```

What it does: Gets keys, values, or entries from objects.

Optional Chaining (?.)

Syntax:

```
obj?.prop
arr?.[0]
fn?.()
```

Example:

```
const name = user?.profile?.name;
```

What it does: Safely accesses nested values.

Nullish Coalescing (??)

Syntax:

```
value ?? fallback
```

Example:

```
const name = input ?? "Guest";
```

What it does: Uses fallback only if value is null or undefined.

Short-Circuit Evaluation

Syntax:

```
a && b
a || b
```

Example:

```
const show = isOpen && <Modal />;
```

What it does: Returns b if a is truthy (a && b), or a if falsy (a || b).

typeof and instanceof

Syntax:

```
typeof value  
value instanceof Type
```

Example:

```
typeof "hi" // "string"  
arr instanceof Array // true
```

What it does: Checks value type or class.

Template Literals

Syntax:

```
‘string ${expression}‘
```

Example:

```
‘Hello, ${name}!’
```

What it does: Embeds variables/expressions in strings.

Default Parameters

Syntax:

```
function fn(a = value) {}
```

Example:

```
function greet(name = "Guest") {}
```

What it does: Sets fallback values for function arguments.

Export Named/Default

Syntax:

```
export default value;  
export { name1, name2 };
```

Example:

```
export default App;  
export { add, subtract };
```

What it does: Shares code for use in other files.

Enumerating Objects

Syntax:

```
for (let key in obj) { ... }  
for (let val of arr) { ... }
```

Example:

```
for (let k in person) { console.log(k); }  
for (let n of nums) { console.log(n); }
```

What it does: Loops over object keys or array values.

Try / Catch

Syntax:

```
try {  
    // code  
} catch (err) {  
    // handle error  
}
```

Example:

```
try {  
    JSON.parse(str);  
} catch (e) {  
    alert("Bad JSON");  
}
```

What it does: Handles errors in code blocks.

Switch Statement

Syntax:

```
switch (value) {  
    case a: ...; break;  
    case b: ...; break;  
    default: ...;  
}
```

Example:

```
switch (color) {  
    case "red": alert("Stop"); break;  
    default: alert("Go");  
}
```

What it does: Runs code based on value matching.

Memoization (useMemo)

Syntax:

```
const memoized = useMemo(() => compute(), [deps]);
```

Example:

```
const doubled = useMemo(() => n * 2, [n]);
```

What it does: Caches expensive calculations in React.

Callback Memoization (useCallback)

Syntax:

```
const fn = useCallback(() => { ... }, [deps]);
```

Example:

```
const handleClick = useCallback(() => setCount(c => c+1), []);
```

What it does: Caches functions to avoid unnecessary re-renders.

Refs (useRef)

Syntax:

```
const ref = useRef(initialValue);  
<element ref={ref} />
```

Example:

```
const inputRef = useRef();  
<input ref={inputRef} />
```

What it does: Accesses DOM nodes or stores mutable values.

Context (useContext)

Syntax:

```
const value = useContext(MyContext);
```

Example:

```
const theme = useContext(ThemeContext);
```

What it does: Reads shared data from React context.

Custom Hooks

Syntax:

```
function useSomething() {  
  // use hooks  
  return value;  
}
```

Example:

```
function useCounter() {  
  const [n, setN] = useState(0);  
  return [n, setN];  
}
```

What it does: Encapsulates reusable logic with hooks.

PropTypes (Runtime Validation)

Syntax:

```
Component.propTypes = { prop: PropTypes.type }
```

Example:

```
Greeting.propTypes = { name: PropTypes.string }
```

What it does: Checks prop types at runtime (optional).

Exporting Multiple Values

Syntax:

```
export { a, b, c };
```

Example:

```
export { add, subtract };
```

What it does: Shares several values from a module.

Spread Props

Syntax:

```
<Component {...props} />
```

Example:

```
<MyButton {...buttonProps} />
```

What it does: Passes all props at once to a component.

Key Prop (for Lists)

Syntax:

```
<element key={uniqueValue} />
```

Example:

```
{items.map(i => <li key={i.id}>{i.text}</li>))}
```

What it does: Helps React track list items efficiently.

Children Prop

Syntax:

```
function Comp(props) { props.children }  
<Comp>Content</Comp>
```

Example:

```
function Wrapper(props) {  
  return <div>{props.children}</div>;  
}
```

What it does: Lets components wrap other elements/content.